DSA0310 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 3

DESIGN, PARSE AND PROBABILISTICALLY RANK ENGLISH QUERIES FOR A UNIVERSITY HELPDESK

Name: MONISHAVARN V
REGNO: 192424398

CONTENTS

1. Task 1 – Context-Free Grammar
2. Task 2 – Derivations and Parse Trees
3. Task 3 – Sentence-Level Constructions
4. Task 4 – Agreement
5. Task 5 – Subcategorization
6. Task 6 – Top-Down Parsing
7. Task 7 – Earley Parsing
8. Task 8 – Feature Structures and Unification
9. Task 9 – Probabilistic CFG
10. Task 10 – Implementation and Testing
11. Task 11 – Comparison and Recommendation
12. Task 12 – Day-4 Design Challenge
13. Conclusion

Note: The assignment sheet specifies a designed grammar, a 30-query test set, a 20-sentence mini-treebank and parser measurements. The worked grammar, examples and hand calculations below form a reproducible academic design. Runtime/state figures are illustrative analytical estimates, not results from executing code on a particular computer.

TASK 1 – DESIGN A CONTEXT-FREE GRAMMAR FOR ENGLISH SYNTAX (5 MARKS)

1.1 Formal definition

A context-free grammar is G = (N, Σ, R, S), where N is the set of non-terminal symbols, Σ is the terminal vocabulary, R is the set of production rules and S is the start symbol. For this helpdesk fragment, S is the start symbol. Non-terminals include S, DECL, IMP, YNQ, WHQ, NP, VP, PP, AP, CP, Det, N, V, Aux, Pron, Adj, Adv, Prep and Conj. Terminals are the words listed in the lexicon.

1.2 Phrase-structure rules (CFG)

S → DECL | IMP | YNQ | WHQ

DECL → NP VP | NP Aux VP

IMP → VP

YNQ → Aux NP VP

WHQ → Wh NP VP | Wh Aux NP VP | Wh VP

NP → Det N | Det Adj N | Pron | ProperN | NP PP | NP RelClause | NP Conj NP

VP → V | V NP | V NP NP | V PP | V NP PP | V CP | V NP CP | Aux VP | Adv VP | VP PP | VP Conj VP

PP → Prep NP

AP → Adj | Adv Adj | AP Conj AP

CP → Comp DECL | Comp WHQ

RelClause → RelPron VP | RelPron Aux VP

Det → the | a | an | this | that | these | those | my | your

N → student | students | staff | timetable | exam | exams | hostel | room | fee | fees | library | book | form | portal | receipt | notice | placement | seminar | result | course | application | department | date | account | scholarship | password | campus | laboratory | certificate | bed

V → show | book | submit | send | tell | check | apply | pay | display | update | need | find | open | close | crash | arrive | postpone | schedule | borrow | return | register | cancel | explain | print | access | confirm | have | is | are | was | were

Prep → with | near | for | to | in | on | at | from | about | before | after

Adj → late | available | current | final | hostel | exam | new | important | postponed | unpaid | online | academic

Adv → currently | quickly | recently | tomorrow | today

Pron → I | we | you | he | she | they | me | us | him | her | them

ProperN → Library | Registrar | Admissions | Chennai

Aux → do | does | did | can | could | will | would | is | are | was | were

Wh → who | what | when | where | why | which | how

Comp → that | whether | if

RelPron → that | which | who

Conj → and | or | but

The expanded lexical productions are grouped by part of speech for readability. Together with the phrase rules they provide more than 40 individual lexical/phrase productions. The lexicon is deliberately domain-focused and contains over 60 distinct words.

1.3 Lexicon and category choice

Category	**Representative words**
Determiners	the, a, an, this, that, these, those, my, your
Nouns	student, students, staff, timetable, exam, exams, hostel, room, fee, fees, library, book, form, portal, receipt, notice,

	placement, seminar, result, course, application, department, date, account, scholarship, password, campus, laboratory, certificate, bed
Verbs	show, book, submit, send, tell, check, apply, pay, display, update, need, find, open, close, crash, arrive, postpone, schedule, borrow, return, register, cancel, explain, print, access, confirm
Prepositions	with, near, for, to, in, on, at, from, about, before, after
Adjectives / adverbs	late, available, current, final, hostel, exam, new, important, postponed, unpaid, online, academic; currently, quickly, recently, tomorrow, today
Pronouns / auxiliaries	I, we, you, he, she, they, me, us, him, her, them; do, does, did, can, could, will, would, is, are, was, were
Other	and, or, but; who, what, when, where, why, which, how; that, whether, if; Library, Registrar, Admissions, Chennai

1.4 Constituency, head and phrasal category

A constituent is a group of words that behaves as a unit. The head determines the basic category and central meaning of a phrase: the noun is the head of NP, the verb is the head of VP, and the preposition is the head of PP.

Sentence	Test	Result
The student checked the timetable.	Substitution: “The student checked it.”	“The timetable” is an NP and can be replaced by a pronoun.
The student checked the timetable near the library.	Movement: “Near the library, the student checked the timetable.”	“Near the library” behaves as a PP constituent.
The student and the staff member checked the timetable.	Coordination: “the student and the staff member”	Two NPs coordinate as one larger NP.

The grammar is context-free because every production replaces one non-terminal independently of its surrounding symbols. A challenging English phenomenon is cross-serial dependency, found in some languages and constructions, which is not naturally represented by a simple CFG. Even in English, long-distance agreement and dependencies require additional features or special rules.

TASK 2 – CONTEXT-FREE RULES, DERIVATIONS AND PARSE TREES (3 MARKS)

2.1 Five example sentences

1. The student checks the timetable.

2. The staff member submits the form.

3. Show the current timetable.

4. Can the student book a room?

5. Which exam did the student check?

2.2 Leftmost and rightmost derivation (Sentence 1)

Sentence: The student checks the timetable.

Leftmost derivation:

S ⇒ DECL ⇒ NP VP ⇒ Det N VP ⇒ the N VP ⇒ the student VP
⇒ the student V NP ⇒ the student checks NP ⇒ the student checks Det N
⇒ the student checks the timetable

Rightmost derivation:

S ⇒ DECL ⇒ NP VP ⇒ NP V NP ⇒ NP V Det N ⇒ NP V Det the?
Correct rightmost sequence: S ⇒ DECL ⇒ NP VP ⇒ NP V NP ⇒ NP V Det N
⇒ NP V Det timetable ⇒ NP V the timetable ⇒ Det N checks the timetable
⇒ Det student checks the timetable ⇒ the student checks the timetable

Bracketed tree:

[S [DECL [NP [Det The] [N student]] [VP [V checks] [NP [Det the] [N timetable]]]]]

Tree diagram (indented):

```
S
└── DECL
    ├── NP
    │   ├── Det: The
    │   └── N: student
    └── VP
        ├── V: checks
        └── NP
```

├── Det: the
└── N: timetable

For this representation, height = 5 edges from S to a terminal on the longest branch (or 6 levels if levels are counted including the root); nodes = 11; rule applications = 10 when lexical expansions are counted. Tree height conventions should be stated because some texts count nodes/levels rather than edges.

2.3 Four further derivation summaries

Sentence	Main derivation pattern	Parse structure
The staff member submits the form.	S→DECL→NP VP; NP→Det N; VP→V NP; object NP→Det N	[S [NP The staff member] [VP submits [NP the form]]]
Show the current timetable.	S→IMP→VP→V NP; NP→Det Adj N	[S [IMP [VP Show [NP the current timetable]]]]
Can the student book a room?	S→YNQ→Aux NP VP; VP→V NP	[S [YNQ Can [NP the student] [VP book [NP a room]]]]
Which exam did the student check?	S→WHQ→Wh Aux NP VP; VP→V NP	[S [WHQ Which [NP exam] did [NP the student] [VP check __]]]

A derivation is a sequence of rule applications; a parse tree records the hierarchical result. Different expansion orders can produce the same tree, so multiple derivations may correspond to one tree.

2.4 Ambiguity

Example 1: “Show the students with the late fee notice.” The PP “with the late fee notice” may attach to the noun students (the students who have the notice) or to the verb show (show them using/along with the notice).

[S [IMP [VP Show [NP [NP the students] [PP with the late fee notice]]]]]

[S [IMP [VP [VP Show [NP the students]] [PP with the late fee notice]]]]

Example 2: “Book the room near the library for the seminar.” “Near the library” can modify room or book; “for the seminar” can attach to the VP or the room NP, creating further readings.

[VP Book [NP [NP the room] [PP near the library]] [PP for the seminar]]

[VP [VP Book [NP the room]] [PP near the library] [PP for the seminar]]

Membership test: a parser starts with S and attempts to derive the exact input token sequence using productions. If a complete derivation consumes every token, the string

belongs to L(G). A CFG may overgenerate or undergenerate and cannot, by itself, verify meaning or real-world truth.

TASK 3 – SENTENCE-LEVEL CONSTRUCTIONS (4 MARKS)

Construction	Rule / example	Structural note
Declarative	DECL → NP VP; "The student checks the timetable."	Subject precedes predicate.
Imperative	IMP → VP; "Show the timetable."	Subject 'you' is understood and normally omitted.
Yes/no question	YNQ → Aux NP VP; "Can the student book a room?"	Auxiliary moves before the subject.
Wh-question (object)	WHQ → Wh Aux NP VP; "Which exam did the student check?"	Wh-object is fronted; object gap is shown in VP.
Wh-question (subject)	WHQ → Wh VP; "Who checked the timetable?"	Wh phrase is the subject; do-support is not required.
Embedded / subordinate	CP → Comp DECL or Comp WHQ; "Tell me whether the fee is paid."	A clause occurs inside a larger VP.
Relative clause	RelClause → RelPron Aux VP; "the exams that were postponed"	Relative clause modifies a noun.

Parse sketches:

Declarative: [S [DECL [NP The student] [VP checks [NP the timetable]]]]

Imperative: [S [IMP [VP Show [NP the timetable]]]]

Yes/no: [S [YNQ Can [NP the student] [VP book [NP a room]]]]

Wh: [S [WHQ Which exam] did [NP the student] [VP check __]]

Coordination combines like categories, for example NP → NP Conj NP for "fees and hostel charges", and VP → VP Conj VP for "check the result and print the receipt". Long-distance dependencies such as "Which form did the officer say that the student submitted?" require gap threading or feature passing. Each construction adds rules and increases ambiguity, especially when clauses and coordinations can attach at multiple levels.

TASK 4 – AGREEMENT (3 MARKS)

Agreement includes subject–verb person/number agreement, determiner–noun number agreement and pronoun case. A simple split-category grammar may use S→NP-sg VP-

sg | NP-pl VP-pl. If the grammar has 2 number values, 3 person values, 2 cases and 2 genders, a naïve cross-product creates 2×3×2×2 = 24 feature combinations for each relevant category. If both NP and VP variants are separately enumerated, the grammar grows rapidly; for example, 24 subject categories paired with 24 predicate categories can require up to 24 agreement pairings.

Feature-based alternative: annotate each phrase with NUM, PER and CASE, then require matching values. This avoids listing every combination as a separate symbol.

Test type	**Sentence**	**Baseline CFG**	**Agreement-aware**
Grammatical	The student submits the form.	Accept	Accept
Grammatical	The students submit the forms.	Accept	Accept
Grammatical	She checks the timetable.	Accept	Accept
Grammatical	They check the timetable.	Accept	Accept
Grammatical	These students pay the fee.	Accept	Accept
Grammatical	I send them the receipt.	Accept	Accept
Ungrammatical	The students submits the form.	May accept	Reject
Ungrammatical	These student pays the fee.	May accept	Reject
Ungrammatical	Me checks the timetable.	May accept	Reject
Ungrammatical	The student submit the form.	May accept	Reject
Ungrammatical	Those book is available.	May accept	Reject
Ungrammatical	Him send the receipt.	May accept	Reject

TASK 5 – SUBCATEGORIZATION (4 MARKS)

Subcategorization (valence) specifies the complements a verb requires or permits. Verbs have different frames:

Frame	**Pattern**	**Example**
Intransitive	V	The portal crashed.

Transitive	V NP	Submit the form.
Ditransitive	V NP NP	Send me the receipt.
Sentential complement	V NP CP	Tell me that the fee is paid.
PP complement	V PP	Apply for a hostel room.
Transitive + PP	V NP PP	Show the student the timetable near the library.

A basic category-based encoding uses V-intr, V-trans, V-ditrans, V-comp and V-pp with matching VP rules. Its drawback is category proliferation when a verb has more than one frame. A lexical SUBCAT feature is more economical.

Complement versus adjunct: a complement is selected by the verb and helps complete its valence ("apply for a room"); an adjunct adds optional information ("apply online yesterday"). Removing a selected complement may make the sentence incomplete, while adjuncts can usually be removed or repeated.

Input	**Expected result**	**Reason**
Submit the form.	Accept	Transitive verb has its NP complement.
The portal crashed.	Accept	Intransitive frame.
Send me the receipt.	Accept	Ditransitive frame.
Tell me that the fee is paid.	Accept	NP plus clause complement.
Apply for a hostel room.	Accept	Selected PP complement.
Submit.	Reject in this intended transitive use	Required NP complement is missing.
The portal crashed the form.	Reject	Crash is used intransitively in this lexicon; object is not licensed.

A verb with multiple frames can create more than one candidate structure. The parser must check each licensed frame, which increases ambiguity and search cost.

TASK 6 – TOP-DOWN PARSING (4 MARKS)

A top-down parser begins with the start symbol S, expands non-terminals using grammar rules, and matches terminals against the input from left to right. When a choice fails, it backtracks to try another production.

6.1 Pseudocode

```
PARSE(symbol, position):
  if symbol is a terminal:
    if input[position] == symbol: return position + 1
```

```
    else: return FAILURE
  for each production symbol -> X1 ... Xk:
    p = position; children = []
    for each Xi from left to right:
      result = PARSE(Xi, p)
      if result is FAILURE: abandon this production
      else: p = result.end; append result
    if all Xi succeed: return completed tree and p
  return FAILURE
Accept only if PARSE(S, 0) consumes all input tokens.
```

6.2 Trace: unambiguous example

Input: “The student checks the timetable.”

Step	Goal / stack (top first)	Action
1	S	Expand S→DECL.
2	DECL	Expand DECL→NP VP.
3	NP	Expand NP→Det N.
4	Det, N	Match ‘The’, then ‘student’.
5	VP	Expand VP→V NP.
6	V, NP	Match ‘checks’; expand NP→Det N.
7	Det, N	Match ‘the’, then ‘timetable’.
8	Empty	All tokens consumed: accept.

6.3 Trace: ambiguous example

Input: “Show the students with the notice.” The parser first tries NP→NP PP and attaches “with the notice” to “students”. If the parse is completed, it may backtrack to the VP-level option VP→VP PP and produce a second valid tree. Both trees are retained if all parses are requested.

Left recursion problem: NP→NP PP calls NP again without consuming a word, so naïve recursive descent can loop indefinitely. Eliminate direct left recursion by rewriting NP→BaseNP NP-PP-Tail and NP-PP-Tail→PP NP-PP-Tail | ε. This preserves the intended repeated-PP structures but changes the intermediate tree shape; a semantic reconstruction step can restore the original grouping.

With k potentially attachable PPs, there can be many attachment combinations (often exponential in k, depending on grammar and constraints). Naïve backtracking can repeatedly parse the same spans. Memoization or chart parsing avoids recomputing identical subproblems. General CFG parsing has a cubic-time worst-case bound for standard chart methods; naïve recursive descent can be exponential.

TASK 7 – EARLEY PARSING (5 MARKS)

An Earley state is written [A→α • β, i], where the dot marks the portion already recognized and i is the input position where this constituent began. The chart S[0]…S[n] stores states ending at each input position.

- Predictor: when the dot is before a non-terminal B, add B productions to the current chart.
- Scanner: when the dot is before a terminal matching the next input word, advance the dot into the next chart.
- Completer: when a state completes a constituent, advance states in its origin chart that were waiting for that constituent.

Worked input (six words): "The student checks the current timetable." Token positions: 0 The, 1 student, 2 checks, 3 the, 4 current, 5 timetable, 6 end. The following compact chart lists the key states; lexical preterminals are shown explicitly.

Chart	**Key states added (dot position shown)**	**Operation**
S[0]	[S→•DECL,0]; [DECL→•NP VP,0]; [NP→•Det N,0]; [Det→•The,0]	Predictor
S[1]	[Det→The•,0]; [NP→Det•N,0]; [N→•student,1]	Scanner + Predictor
S[2]	[N→student•,1]; [NP→Det N•,0]; [DECL→NP•VP,0]; [VP→•V NP,2]; [V→•checks,2]	Completer + Predictor
S[3]	[V→checks•,2]; [VP→V•NP,2]; [NP→•Det Adj N,3]; [Det→•the,3]	Scanner + Predictor
S[4]	[Det→the•,3]; [NP→Det•Adj N,3];	Scanner + Predictor

	[Adj→•current,4]	
S[5]	[Adj→current•,4]; [NP→Det Adj•N,3]; [N→•timetable,5]	Scanner + Predictor
S[6]	[N→timetable•,5]; [NP→Det Adj N•,3]; [VP→V NP•,2]; [DECL→NP VP•,0]; [S→DECL•,0]	Completer

The final completed S state in S[6] confirms acceptance. A full implementation retains every predicted and completed state, including duplicate checks; the table above is a readable trace of the essential states rather than every lexical alternative in the expanded lexicon.

Back-pointers record which child states produced a completed state. For an ambiguous PP attachment, both the NP-attached and VP-attached back-pointer combinations can be stored in a packed parse forest without duplicating shared subtrees. Earley handles left recursion because a state is added to a chart only once; repeated prediction does not create an infinite loop.

For a general CFG, Earley parsing has $O(n^3)$ worst-case time and $O(n^2)$ chart-space complexity (grammar-dependent improvements are possible). Unlike simple recursive descent, it shares partial results. On the six-word example, the compact trace has 6 chart columns and a small set of states; a recursive-descent parser may revisit the same NP/PP spans during backtracking. Exact state counts depend on the full grammar and implementation.

TASK 8 – FEATURE STRUCTURES AND UNIFICATION (4 MARKS)

A feature structure is an attribute–value matrix. A feature path is a sequence of attributes used to access a value (for example, [SYNSEM AGR NUM]). Reentrancy means two paths share the same value or structure. Unification combines compatible feature structures; conflicting values cause failure. Subsumption is a general-to-specific relation: a more general structure contains fewer constraints.

8.1 Lexical feature entries

Word	Category	Features
student	N	NUM=sg, PERSON=3, CASE=nom/acc
students	N	NUM=pl, PERSON=3
checks	V	AGR=(3,sg),

		SUBCAT=<NP>
submit	V	SUBCAT=<NP>
send	V	SUBCAT=<NP,NP>
apply	V	SUBCAT=<PP(for)>
me	Pron	PERSON=1, NUM=sg, CASE=acc
I	Pron	PERSON=1, NUM=sg, CASE=nom

8.2 Unification examples

- U1: [NUM=sg, PER=3] ⊔ [NUM=sg] = [NUM=sg, PER=3] (success).
- U2: [CASE=nom, PER=1] ⊔ [CASE=nom] = [CASE=nom, PER=1] (success).
- U3: [NUM=sg] ⊔ [NUM=pl] = failure because the NUM values conflict.

8.3 Feature-constrained rules

S[AGR=a] → NP[AGR=a] VP[AGR=a]

NP[NUM=n] → Det[NUM=n] N[NUM=n]

VP[AGR=a] → V[AGR=a, SUBCAT=< >]

VP[AGR=a] → V[AGR=a, SUBCAT=<NP>] NP

VP[AGR=a] → V[AGR=a, SUBCAT=<NP,NP>] NP NP

VP[AGR=a] → V[AGR=a, SUBCAT=<PP(for)>] PP[HEAD=for]

NP[CASE=nom] → Pron[CASE=nom]

NP[CASE=acc] → Pron[CASE=acc]

For “The student checks the timetable”, the subject NP has AGR=(3,sg), and the verb checks has AGR=(3,sg); unification succeeds. The object NP is assigned accusative/object position. For “The students checks the timetable”, the subject AGR=(3,pl) conflicts with the verb AGR=(3,sg), so the parse fails.

Feature structures replace many split-category rules with a small number of general rules and constraints. They also encode verb valence using SUBCAT. In an Earley parser, unification can be performed when a constituent completes and combines with a waiting state. This adds feature-combination overhead; worst-case complexity can rise with the number and size of feature structures, although chart sharing remains useful.

TASK 9 – PROBABILISTIC CONTEXT-FREE GRAMMARS (5 MARKS)

A PCFG assigns each production A→β a conditional probability P(A→β | A). For every left-hand-side non-terminal A, the probabilities of all its alternative rules must sum to 1.

9.1 Mini-treebank and maximum-likelihood estimates

A 20-sentence hand-annotated mini-treebank can be used. The following compact illustrative count table demonstrates maximum-likelihood estimation; counts are assumed treebank counts for this worked model and should be replaced by actual annotation counts if a separately annotated corpus is collected.

Rule	Count	LHS total	MLE probability
S→DECL	12	20	0.60
S→IMP	3	20	0.15
S→YNQ	3	20	0.15
S→WHQ	2	20	0.10
NP→Det N	12	20	0.60
NP→Pron	4	20	0.20
NP→NP PP	4	20	0.20
VP→V NP	10	20	0.50
VP→V	4	20	0.20
VP→V PP	3	20	0.15
VP→VP PP	3	20	0.15

MLE formula: $P(A \rightarrow \beta|A) = \text{Count}(A \rightarrow \beta) / \Sigma\gamma\ \text{Count}(A \rightarrow \gamma)$. The probabilities in each LHS group sum to 1. For unseen rules, add-one smoothing may be used: $P=(\text{count}+1)/(\text{total}+V)$, where V is the number of alternative rules for that LHS. Smoothing prevents zero probability but can assign probability to implausible rules, so grammar constraints should remain in place.

9.2 PP attachment probability

For “Show the students with the notice”, compare two simplified trees:

T1 (NP attachment): $P(S \rightarrow IMP) \times P(IMP \rightarrow VP) \times P(VP \rightarrow V\ NP) \times P(NP \rightarrow NP\ PP)$
$= 1.0 \times 1.0 \times 0.50 \times 0.20 = 0.10$

T2 (VP attachment): $P(S \rightarrow IMP) \times P(IMP \rightarrow VP) \times P(VP \rightarrow VP\ PP) \times P(VP \rightarrow V\ NP)$
$= 1.0 \times 1.0 \times 0.15 \times 0.50 = 0.075$

Under these illustrative probabilities, T1 is preferred. The sentence probability is the sum of the probabilities of its distinct parses: $P(\text{sentence})=0.10+0.075=0.175$. The calculation is a simplified fragment-level example; a full tree probability multiplies every rule probability, including lexical productions.

9.3 Probabilistic CKY / Viterbi

Convert the grammar to Chomsky Normal Form (CNF): productions are A→BC or A→word, with start-symbol handling for ε if required. Example transformations:

VP→V NP is already binary; DECL→NP VP is binary; NP→Det Adj N becomes NP→Det X and X→Adj N; unary chains are eliminated or handled separately.

```
Initialize V[i,i+1,A] = P(A→word_i) for lexical rules.
For span length 2..n:
  for each span (i,j), each split k, and each A→B C:
    candidate = P(A→B C) * V[i,k,B] * V[k,j,C]
    keep the maximum candidate and its back-pointer.
Return V[0,n,S] and follow back-pointers for the best tree.
```

Short CKY illustration for "students check forms": token spans first create lexical entries N(students), V(check), N(forms). Binary combination creates NP from the first noun if the grammar permits bare plural NPs, VP from V+NP, then S from NP+VP. The best score is the product of the selected rule probabilities. The exact numeric CKY table depends on the CNF rule probabilities; the above recurrence gives the reproducible calculation.

PCFG assumptions and limitations: (1) context-freeness: rule probability ignores wider context; the same VP rule receives the same probability in different topics; (2) position invariance: a rule probability does not depend on sentence position; (3) tree preference: products of many probabilities can favour shorter trees; (4) limited lexical sensitivity: ordinary PCFGs do not directly model word-specific attachment preferences. Lexicalized PCFGs or neural rerankers can address some limitations.

TASK 10 – IMPLEMENT AND TEST THE PARSERS (3 MARKS)

10.1 Python implementation (NLTK)

The following code provides a small runnable demonstration of CFG parsing with NLTK. It can be expanded with the complete rule list and a feature-checking layer. The Earley chart parser is supplied by NLTK; a Viterbi parser requires a PCFG in CNF.

```
# Install once in Google Colab:
# !pip -q install nltk

import nltk
from nltk import CFG
from nltk.parse import RecursiveDescentParser, EarleyChartParser

grammar = CFG.fromstring('''
```

```
S -> DECL | IMP | YNQ
DECL -> NP VP
IMP -> VP
YNQ -> AUX NP VP
NP -> DET N | DET ADJ N | PRON
VP -> V NP | V
DET -> 'the' | 'a'
ADJ -> 'current' | 'late'
N -> 'student' | 'students' | 'timetable' | 'form'
V -> 'checks' | 'check' | 'submit' | 'show'
AUX -> 'can' | 'does'
PRON -> 'they' | 'she'
''')

tests = [
    "the student checks the timetable",
    "the students check the form",
    "show the current timetable",
    "can the student submit the form",
    "the student submit the form"  # agreement layer should reject
]

td = RecursiveDescentParser(grammar)
earley = EarleyChartParser(grammar)

for sent in tests:
    tokens = sent.split()
    print("\\nINPUT:", sent)
    for name, parser in [('Top-down', td), ('Earley', earley)]:
        trees = list(parser.parse(tokens))
        print(name, "accepted:", bool(trees), "parse_count:", len(trees))
        if trees:
            print(trees[0])
```

Important: the demonstration CFG is intentionally small and does not enforce all agreement or subcategorization constraints. To enforce them, add a feature unification

layer or use a feature-based grammar. A PCFG parser can be created with nltk.ViterbiParser after converting the productions to a valid PCFG/CNF grammar.

10.2 Thirty-sentence test set

No.	Test sentence	Type
1	The student checks the timetable.	Declarative
2	The students check the timetable.	Declarative
3	The staff member submits the form.	Declarative
4	The portal crashed.	Declarative
5	The library opens today.	Declarative
6	She pays the hostel fee.	Declarative
7	They borrow a book.	Declarative
8	I need the current result.	Declarative
9	The officer updates the account.	Declarative
10	The exam was postponed.	Declarative
11	Show the timetable.	Imperative
12	Submit the application.	Imperative
13	Check the library notice.	Imperative
14	Book a room for the seminar.	Imperative
15	Print the receipt.	Imperative
16	Can the student book a room?	Yes/no
17	Does the portal show the result?	Yes/no
18	Will the library open tomorrow?	Yes/no
19	Did the staff update the timetable?	Yes/no
20	Could you check my application?	Yes/no
21	When will the exams begin?	Wh-question
22	Where is the library?	Wh-question
23	Which form did the student submit?	Wh-question
24	Who checked the timetable?	Wh-question

25	Why was the exam postponed?	Wh-question
26	Show the students with the late fee notice.	Ambiguous
27	Book the room near the library for the seminar.	Ambiguous
28	I saw the student with the receipt.	Ambiguous
29	The officer discussed the course with the staff.	Ambiguous
30	Find the book on the table in the library.	Ambiguous

10.3 Comparative result format

The table below gives expected qualitative outcomes for the designed grammar. Exact runtime and state counts must be measured by running the code on the target machine; they are not invented here.

Parser	Coverage / output	Ambiguity	Cost profile
Top-down recursive descent	Finds parses for covered sentences; may fail or loop with left recursion unless transformed	May return first parse or enumerate alternatives with backtracking	Can repeat work; worst-case exponential in naïve form
Earley chart parser	Handles left recursion and CFG ambiguity	Can retain all parses in a packed forest	$O(n^3)$ worst-case time, $O(n^2)$ chart space
PCFG / Viterbi	Returns highest-probability parse when grammar is probabilistic and CNF-compatible	Ranks parses; Viterbi returns best one	CKY-style $O(n^3)$ time and $O(n^2)$ space for fixed grammar

Recommended experiment: record for every sentence the accepted flag, number of parses, chart-state count or recursive calls, and elapsed time using Python time.perf_counter(). Plot sentence length against state count. Include the ambiguous examples, malformed agreement examples, and a 20-word sentence. Use the same machine and grammar for a fair comparison.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE PARSING DESIGN (2 MARKS)

Criterion	Top-down	Earley	PCFG / Viterbi

Coverage	Depends on grammar and search order	Strong CFG coverage	Coverage depends on PCFG grammar
Ambiguity	Backtracking may enumerate parses	Stores alternatives in chart/forest	Ranks parses by probability
Efficiency	Can repeat work; left recursion risk	Shares partial parses	Dynamic programming; cubic worst case
Maintenance	Simple rules, but agreement rules multiply	Works with CFG and can be extended with features	Needs annotated treebank and probability updates
Best use	Small controlled queries	Reliable syntactic parsing	Selecting likely interpretation

Recommendation: deploy a feature-based CFG with an Earley parser for syntactic coverage, followed by PCFG ranking for ambiguous parses. Features enforce agreement and subcategorization; Earley avoids left-recursion loops and shares partial results; PCFG ranks candidate trees. The design is more robust than a top-down parser alone, but remains limited by vocabulary coverage, annotation quality and the independence assumptions of PCFGs.

Manual effort includes writing and reviewing at least 40 grammar rules, compiling a 60-word domain lexicon, preparing 30 test queries, and annotating at least 20 parse trees. As the number of message types and exceptions grows, data-driven learning becomes more economical than adding many hand-written rules, provided enough representative annotated data is available.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT (8 MARKS)

A. Impact analysis (2 marks)

The revised helpdesk must accept conversational messages, embedded questions, coordinated clauses, relative clauses, fragments, missing punctuation and partly ungrammatical input. The original design assumed a controlled vocabulary, mostly grammatical single-clause messages and a relatively small ambiguity level. These assumptions no longer hold.

- Embedded question: “Can you tell me when the exams will be held?” requires a WHQ inside a CP complement.

- Relative clause: “the exams that were postponed” requires relative-clause attachment and agreement.
- Coordination: “and also whether the hostel fee is due” requires coordination of clauses or complement phrases.
- Fragments and ellipsis: “Hostel fee deadline?” may lack a finite verb and subject.
- Noise: spelling mistakes, informal wording and missing punctuation may prevent exact lexical matching.

Longer inputs create more spans and more possible attachment points. Earley chart size grows approximately quadratically in input length, while worst-case time grows cubically. The number of parse trees can grow much faster when several PPs, clauses or coordinations are ambiguous.

B. Evaluation of the original design (2 marks)

New message	Original grammar result	First likely failure
Hi, can you tell me when the exams that were postponed will be held?	Partial / reject	Greeting token ‘Hi’ and embedded WH clause are not fully covered.
Is the hostel fee due, and can I pay it online?	Partial / reject	Coordination of two independent questions is not covered by the basic S rules.
Hostel fee deadline?	Reject	Fragment has no finite clause or explicit subject/verb.

The original grammar can parse portions such as “the exams” and “the hostel fee”, but cannot reliably parse the complete conversational messages. A robust partial parser can identify known phrases even when a complete S parse is unavailable. Earley remains more economical than naïve top-down parsing for the expanded CFG because it shares partial states; however, a grammar alone cannot correct spelling or infer omitted words.

C. Adaptation or redesign (2 marks)

Decision: MODIFY the original design rather than discard it. Retain the feature-based CFG and Earley chart parser, then add a robust input-processing and fallback layer.

- Add lexical normalization for case, punctuation, common spelling variants and contractions.
- Add rules for greetings, fragments, embedded questions, relative clauses and coordination of clauses.
- Use feature threading for wh-gaps, subject–verb agreement and clause boundaries.

- Expand the lexicon using helpdesk logs and add domain entities such as course codes, dates and fee names.
- Train a larger PCFG from annotated helpdesk messages; use smoothing for rare constructions.
- If full parsing fails, return a partial parse with recognized entities and confidence rather than rejecting the whole message.

Expected effect: coverage increases because more constructions are represented; ambiguity may also increase because more attachment and coordination options exist. Earley chart sharing controls repeated work, while PCFG ranking selects a likely parse. Annotation effort rises because conversational examples and partial parses must be labelled.

D. Evidence-based defence (2 marks)

The modification is supported by the original analysis: the baseline CFG covers controlled sentence types but fails on greetings, fragments and embedded/coordinated clauses. The Earley algorithm handles left recursion and shares partial results, making it a sound parsing foundation. Feature structures enforce agreement and valence without multiplying categories. PCFG probabilities rank ambiguous parses using observed usage. The revised design therefore improves coverage and interpretation while preserving the reliable components already developed.

Remaining limitations include unseen vocabulary, severe spelling errors, context-dependent meanings, biased or sparse treebank data and the cubic worst-case parsing bound. A future version can add a neural language-model reranker while keeping the grammar parser for explainable structure and constraint checking.

CONCLUSION

This assignment designed a context-free grammar for a university helpdesk, represented common English sentence constructions, and examined derivations, parse trees, agreement and verb subcategorization. Top-down parsing is simple but vulnerable to left recursion and repeated work. Earley parsing uses a chart to handle ambiguity and left recursion efficiently. Feature structures provide a compact method for agreement and valence, while a PCFG ranks multiple valid parses. For conversational helpdesk messages, the recommended solution is a modified feature-based CFG with Earley parsing, probabilistic ranking and a partial-parsing fallback.

REFERENCES

1. Jurafsky, D. and Martin, J. H. (2023). Speech and Language Processing (3rd ed. draft). Chapters on CFGs, parsing and PCFGs.

2. Allen, J. (1995). Natural Language Understanding (2nd ed.). Benjamin/Cummings.

3. Hopcroft, J. E., Motwani, R. and Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.

4. Earley, J. (1970). An efficient context-free parsing algorithm. Communications of the ACM, 13(2), 94–102.

5. Kasami, T. (1965). An efficient recognition and syntax-analysis algorithm for context-free languages. Technical Report, University of Illinois.

6. Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. IEEE Transactions on Information Theory, 13(2), 260–269.

7. Charniak, E. (1997). Statistical parsing with a context-free grammar and word statistics. Proceedings of AAAI.

8. Collins, M. (1999). Head-Driven Statistical Models for Natural Language Parsing. PhD thesis, University of Pennsylvania.

9. Stuart, M. and colleagues. (Course material). Context-free grammars, chart parsing and probabilistic parsing lecture notes.